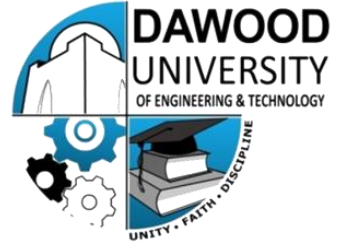


LECTURE – OPERATORS OOP (JAVA)



MR. PREM KUMAR

LECTURER

DEPARTMENT OF CYBER SEURITY

Dawood University of Engineering and Technology, Karachi

AGENDA



- Operators
- Types of Operator
- Keywords

OPERATORS IN JAVA

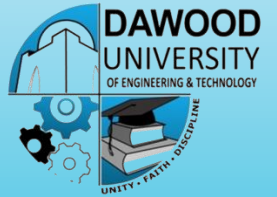
- **Operator** in Java is a symbol that is used to perform operations. For example: +, -, *, / etc.
 - There are many types of operators in Java which stated here.
- Unary Operator,
 - Arithmetic Operator,
 - Shift Operator,
 - Relational Operator,
 - Bitwise Operator,
 - Logical Operator,
 - Ternary Operator and
 - Assignment Operator.

JAVA UNARY OPERATOR

- The Java unary operators require only one operand. Unary operators are used to perform various operations i.e.:
- incrementing/decrementing a value by one
- negating an expression
- inverting the value of a Boolean

```
public class OperatorExample{  
    public static void main(String args[]){  
        int x=10;  
        System.out.println(x++);  
        System.out.println(++x);  
        System.out.println(x--);  
        System.out.println(--x);  
    }  
}
```

JAVA ARITHMETIC OPERATORS



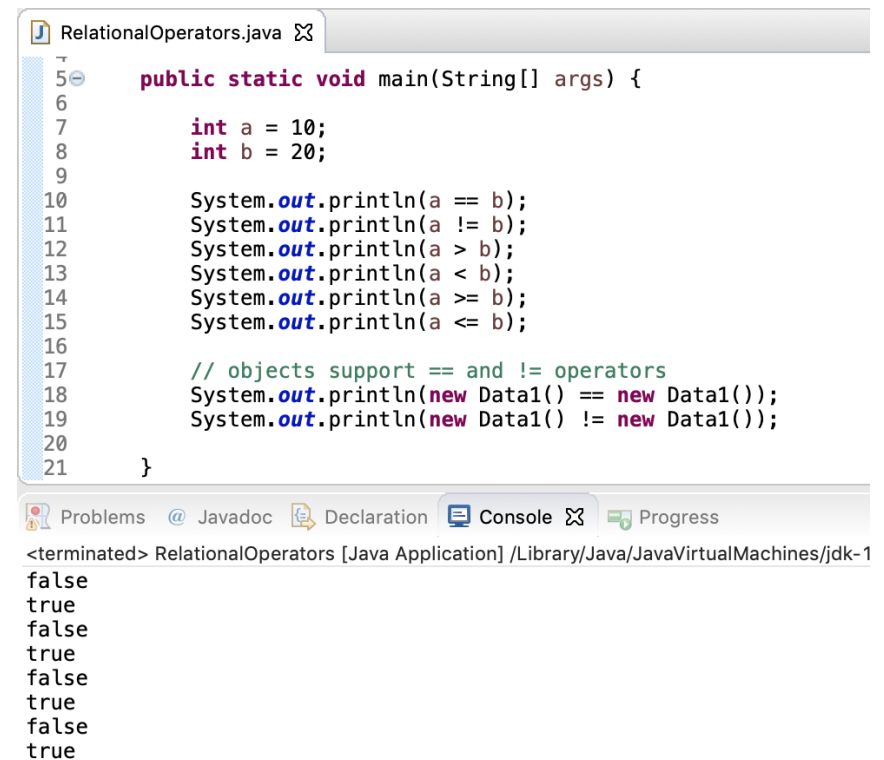
- Java arithmetic operators are used to perform addition, subtraction, multiplication, and division. They act as basic mathematical operations.

RELATIONAL OPERATORS

- Java has 6 relational operators.
 1. `==` is the equality operator. This returns true if both the operands are referring to the same object, otherwise false.
 2. `!=` is for non-equality operator. It returns true if both the operands are referring to the different objects, otherwise false.
 3. `<` is less than operator.
 4. `>` is greater than operator.
 5. `<=` is less than or equal to operator.
 6. `>=` is greater than or equal to operator.

RELATIONAL OPERATORS SUPPORTED DATA TYPES

- The == and != operators can be used with any primitive data types as well as objects.
- The <, >, <=, and >= can be used with primitive data types that can be represented in numbers. It will work with char, byte, short, int, etc. but not with boolean. These operators are not supported for objects.



```
RelationalOperators.java
5 public static void main(String[] args) {
6
7     int a = 10;
8     int b = 20;
9
10    System.out.println(a == b);
11    System.out.println(a != b);
12    System.out.println(a > b);
13    System.out.println(a < b);
14    System.out.println(a >= b);
15    System.out.println(a <= b);
16
17    // objects support == and != operators
18    System.out.println(new Data1() == new Data1());
19    System.out.println(new Data1() != new Data1());
20
21 }
```

Problems Javadoc Declaration Console Progress

<terminated> RelationalOperators [Java Application] /Library/Java/JavaVirtualMachines/jdk-1

false
true
false
true
false
true
false
true

JAVA LEFT SHIFT OPERATOR

- The Java left shift operator << is used to shift all of the bits in a value to the left side of a specified number of times.

```
System.out.println(10<<2);
```

```
System.out.println(10<<3);
```

```
System.out.println(20<<2);
```

```
System.out.println(15<<4);
```

JAVA RIGHT SHIFT OPERATOR

- The Java right shift operator >> is used to move the value of the left operand to right by the number of bits specified by the right operand.

```
System.out.println(10>>2);
```

```
System.out.println(20>>2);
```

```
System.out.println(20>>3);
```

JAVA AND OPERATOR EXAMPLE: LOGICAL && AND BITWISE &

- The logical && operator doesn't check the second condition if the first condition is false. It checks the second condition only if the first one is true.
- The bitwise & operator always checks both conditions whether first condition is true or false.

```
int a=10;  
int b=5;  
int c=20;  
System.out.println(a<b&&a<c);  
System.out.println(a<b&a<c);  
}}
```

```
int a=10;  
int b=5;  
int c=20;  
System.out.println(a<b&&a++<c);  
System.out.println(a);  
System.out.println(a<b&a++<c);  
System.out.println(a);
```

JAVA OR OPERATOR EXAMPLE: LOGICAL || AND BITWISE |

- The logical || operator doesn't check the second condition if the first condition is true. It checks the second condition only if the first one is false.
- The bitwise | operator always checks both conditions whether first condition is true or false.

```
int a=10;
int b=5;
int c=20;
System.out.println(a>b || a<c); //true || true = true
System.out.println(a>b | a<c); //true | true = true
//|| vs |
System.out.println(a>b || a++<c); //true || true = true
System.out.println(a); //10 because second condition is not checked
System.out.println(a>b | a++<c); //true | true = true
System.out.println(a); //11 because second condition is checked
```

TERNARY OPERATOR

- Java Ternary operator is used as one line replacement for if-then-else statement and used a lot in Java programming. It is the only conditional operator which takes three operands.

```
public class OperatorExample{  
    public static void main(String args[]){  
        int a=2;  
        int b=5;  
        int min=(a<b)?a:b;  
        System.out.println(min);  
    }  
}
```

JAVA KEYWORDS



- Java keywords are also known as reserved words. Keywords are particular words that act as a key to a code. These are predefined words by Java so they cannot be used as a variable or object name or class name.
- **Some examples: for, if, break, float, final, class**

Thanks 😊